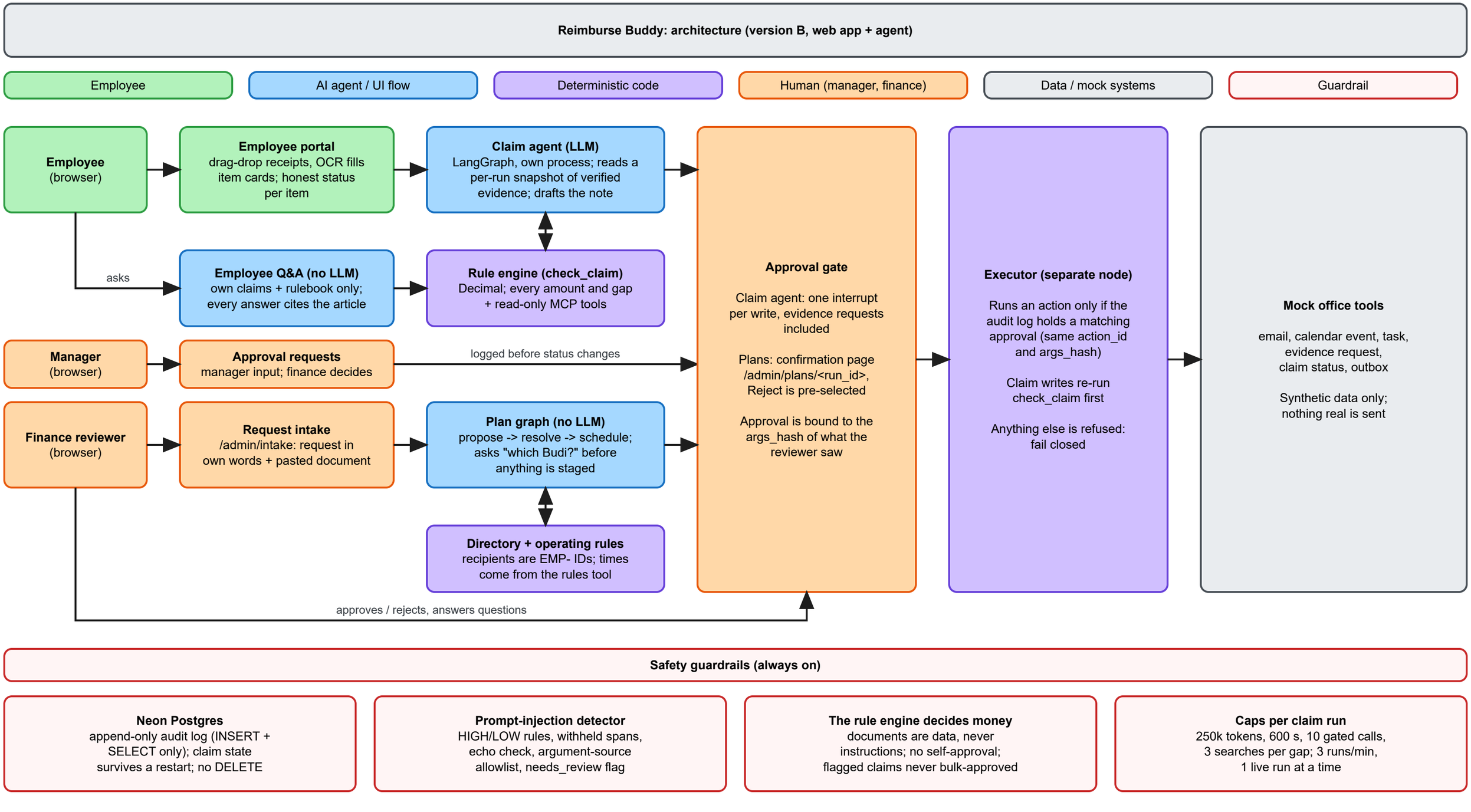

Reimburse Buddy: architecture (version B, web app + agent)
Employee
AI agent / UI flow
Deterministic code
Human (manager, finance)
Data / mock systems
Guardrail
Employee
(browser)
Employee portal
drag-drop receipts, OCR fills item cards; honest status per item
Claim agent (LLM)
LangGraph, own process; reads a per-run snapshot of verified evidence; drafts the note
asks
Employee Q&A (no LLM)
own claims + rulebook only; every answer cites the article
Rule engine (check_claim)
Decimal; every amount and gap + read-only MCP tools
Manager
(browser)
Approval requests
manager input; finance decides
logged before status changes
Finance reviewer
(browser)
Request intake
/admin/intake: request in own words + pasted document
Plan graph (no LLM)
propose -> resolve -> schedule; asks "which Budi?" before anything is staged
Directory + operating rules
recipients are EMP- IDs; times come from the rules tool
Approval gate
Claim agent: one interrupt per write, evidence requests included
Plans: confirmation page /admin/plans/<run_id>, Reject is pre-selected
Approval is bound to the args_hash of what the reviewer saw
approves / rejects, answers questions
Executor (separate node)
Runs an action only if the audit log holds a matching approval (same action_id and args_hash)
Claim writes re-run check_claim first
Anything else is refused: fail closed
Mock office tools
email, calendar event, task, evidence request, claim status, outbox
Synthetic data only; nothing real is sent
Safety guardrails (always on)
Neon Postgres
append-only audit log (INSERT + SELECT only); claim state survives a restart; no DELETE
Prompt-injection detector
HIGH/LOW rules, withheld spans, echo check, argument-source allowlist, needs_review flag
The rule engine decides money
documents are data, never instructions; no self-approval; flagged claims never bulk-approved
Caps per claim run
250k tokens, 600 s, 10 gated calls, 3 searches per gap; 3 runs/min, 1 live run at a time